

```

import crypto from 'node:crypto';
import { q, one } from '../db.js';
import { publicProfile } from './projection.js';
import { listCapabilities, resolveDisposition } from './policy.js';

// Every business has two surfaces.
//
//   public    - no credential. Exactly what the business chose to publish,
//               assembled from installed apps. Menu, hours, availability.
//   private   - a token tied to a membership. Everything that membership could
//               do by hand, offered as tools, going through the same gate.
//
// The private surface is not a superset of the public one by accident: it is
// the same projections plus the capabilities that person is granted.

export async function publicSurface(slug) {
  const business = await one(`select * from business where slug = $1 and status =
  if (!business) return null;

  const sections = await q(
    `select pm.section_key, pm.title, i.package_key
      from projection_map pm join install i on i.id = pm.install_id
     where pm.business_id = $1 and pm.visible = true and i.status = 'active'
     order by pm.sort_order`,
    [business.id]
  );

  return {
    name: `ghost:${business.slug}`,
    surface: 'public',
    description: `Published information for ${business.display_name}. No credenti
    resources: sections.map(s => ({
      uri: `ghost://${business.slug}/${s.section_key}`,
      name: s.title, mimeType: 'application/json', providedBy: s.package_key,
    })),
    tools: [
      ...sections.map(s => ({
        name: `read_${s.section_key}`,
        description: `Read the ${s.title.toLowerCase()} published by ${business.d
        inputSchema: { type: 'object', properties: {}, additionalProperties: fals
      })),
      {
        name: 'check_availability',
        description: `Ask what is open on a date. The search is recorded for ${bu

```

```

    inputSchema: {
      type: 'object',
      properties: { date: { type: 'string' }, partySize: { type: 'integer' }
      required: ['date'],
    },
  },
],
};
}

// The private surface needs a token, and only offers what that membership
// is actually granted. A tool that is not granted does not appear.
export async function privateSurface({ slug, token }) {
  const business = await one(`select * from business where slug = $1 and status =
  if (!business) return null;

  const auth = await authenticate({ businessId: business.id, token });
  if (!auth) return { error: 'unauthorized', surface: 'private' };

  const ctx = { businessId: business.id, business, person: auth.person, membershi

  const tools = [];
  for (const cap of listCapabilities()) {
    const d = await resolveDisposition({
      businessId: business.id, membership: auth.membership, capability: cap.key,
    });
    if (d === 'never') continue;
    tools.push({
      name: cap.key.replace(/\.\/g, '_'),
      capability: cap.key,
      description: cap.summary ?? cap.key,
      disposition: d, // "ask" means it will park for appro
      scriptOnly: cap.agentSafe === false, // still callable; a script runs it,
      inputSchema: cap.input ?? { type: 'object' },
    });
  }

  const canonical = await one(
    `select count(*)::int as facts from business_fact where business_id = $1 and
    [business.id]
  );

  return {
    name: `ghost:${business.slug}:private`,
    surface: 'private',
    description: `Operating surface for ${business.display_name}, as ${auth.membe
    actingAs: { role: auth.membership.role, person: auth.person?.display_name ??

```

```

    resources: [
      { uri: `ghost://${business.slug}/canonical`, name: 'Canonical business reco
        mimeType: 'application/json', note: `${canonical.facts} facts` },
      { uri: `ghost://${business.slug}/drift`, name: 'What each connected app cur
        mimeType: 'application/json' },
      { uri: `ghost://${business.slug}/receipts`, name: 'Receipt ledger', mimeTyp
    ],
    tools,
  };
}

export async function readResource(slug, sectionKey) {
  const profile = await publicProfile(slug);
  if (!profile) return null;
  const section = profile.sections.find(s => s.key === sectionKey);
  if (!section) return null;
  return { uri: `ghost://${slug}/${sectionKey}`, data: section.data };
}

// --- tokens -----

const hash = (t) => crypto.createHash('sha256').update(t).digest('hex');

export async function issueToken({ businessId, membershipId, label, scopes = [] }
  const token = 'ghp_' + crypto.randomBytes(24).toString('hex');
  await one(
    `insert into access_token (business_id, membership_id, label, token_hash, sco
      values ($1,$2,$3,$4,$5::jsonb) returning *`,
    [businessId, membershipId, label, hash(token), JSON.stringify(scopes)]
  );
  return token; // shown once
}

export async function revokeToken({ businessId, tokenId }) {
  return one(
    `update access_token set revoked_at = now() where id = $1 and business_id = $
    [tokenId, businessId]
  );
}

async function authenticate({ businessId, token }) {
  if (!token) return null;
  const row = await one(
    `select * from access_token where business_id = $1 and token_hash = $2 and re
    [businessId, hash(token)]
  );
  if (!row) return null;

```

```
const membership = await one(`select * from membership where id = $1`, [row.mem
if (!membership || membership.status !== 'active') return null;
const person = await one(`select * from person where id = $1`, [membership.pers
return { membership, person, token: row };
}

export { authenticate };
```